Why MCP?

How AI assistants stopped being islands — and why everyone is suddenly speaking the same protocol.

A distilled summary of *Model Context Protocol — The Why* (CampusX, Nitish Singh, 52 min). Written by Claude.

Figure 1 · The title slide. "The Why" is the first of three videos in Nitish's MCP trilogy. This summary covers exactly that question: why does MCP need to exist at all?

In one paragraph

Every AI assistant needs **context** — the documents, code, tickets and conversations around your work — to be useful. For two years we solved that by writing custom "tools" inside every chatbot, one per service, one per chatbot. The math is brutal: N chatbots × M services = N×M integrations, all separately maintained. **MCP** (Anthropic's Model Context Protocol) replaces that with a tiny agreement: speak this protocol, and any compliant chatbot can talk to any compliant tool. The integration count collapses from N×M to N+M, and the heavy lifting moves to the service that owns the tool. That is the whole pitch.

Act 1 — The stage MCP walked onto

To understand why MCP exists, you have to understand the world it found. Nitish frames the story as starting on **November 30, 2022** — the day ChatGPT launched. He is right to start there. Before that date, our 500-year relationship with machines was almost entirely *transactional*: press button, get result. After it, suddenly, we could *talk* to software — not just instruct it.

Adoption rolled out in three waves, each one widening what AI assistants were expected to do:

Wave 1 · Pure wonder

December 2022. Asking ChatGPT to write a Shakespearean sonnet about pizza or explain quantum mechanics from a cat's perspective. Social media was full of screenshots. Nothing productive happened, but curiosity calibrated everyone's mental model of what "talking to software" could feel like.

Wave 2 · Professional adoption

Early 2023. A lawyer drops a 50-page contract into ChatGPT and asks for a summary. A developer pastes a stack trace and asks why. A teacher hands over a syllabus and asks for a curriculum. For the first time, the chatbot is a *colleague*, not a party trick. Productivity bumps measurably. People keep coming back.

Wave 3 · The API revolution

Mid 2023 onwards. OpenAI exposed the GPT models as an API. Microsoft wired Copilot into Word, Excel, PowerPoint. Google added AI to Gmail, Docs, Sheets, Drive. Cursor and Perplexity launched as native AI products. The big shift is that **AI is no longer one app you visit — it is a feature inside every app you already use.**

That third wave is where Nitish's story bends, because it created a problem nobody noticed at first.

Act 2 — Fragmentation: AI everywhere, AI nowhere

Once every app got its own AI, we discovered the catch: **none of them know about each other**. Notion's AI has no idea what's happening in Slack. Slack's AI cannot see your VS Code editor. Your code assistant cannot read the ticket in Jira that explains *why* you're writing this function.

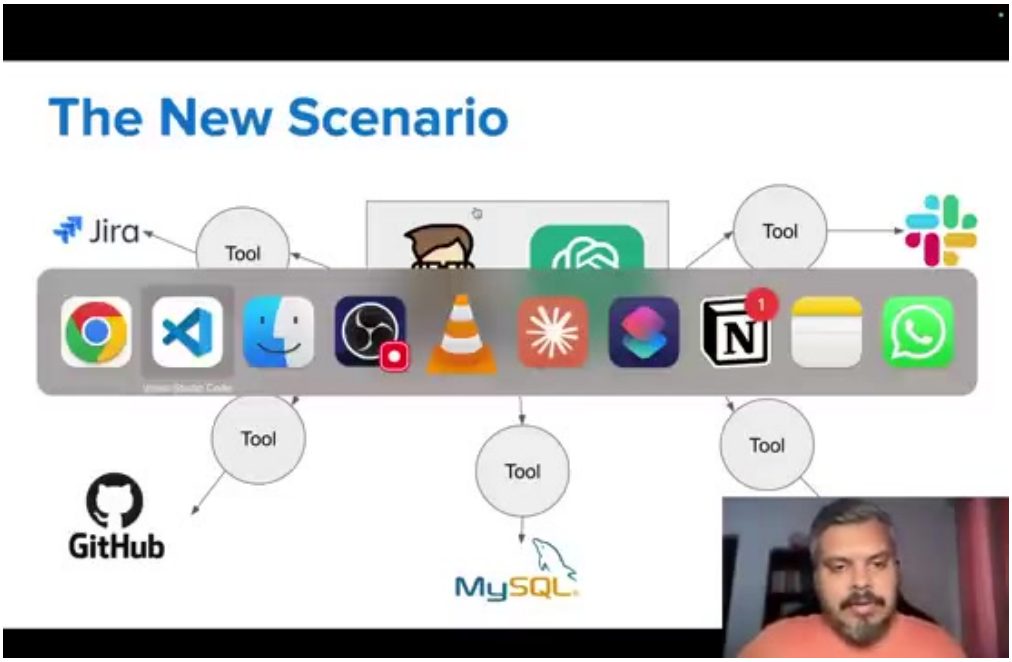

Figure 2 · The fragmentation problem in one image. Each app has its own AI, each AI lives on its own island. The user has to be the glue.

What we actually wanted, from the moment we first saw GPT-4, was *one* agent that could see all of our work end-to-end. What we got instead was five disconnected assistants and a new job: **being the human router between them**.

Nitish gives this a name worth remembering. Behind every fragmentation problem lies a deeper one: a **context** problem.

Act 3 — Why "context" is the load-bearing word

The middle letter of MCP is the one most people skim past. It shouldn't be. **Context is everything an AI can see when it generates a response.** Conversation history, sure — but also: open files, ticket descriptions, database schemas, security guidelines, the discussion in Slack three days ago about why the auth flow looks weird.

When you and ChatGPT discuss quantum physics, context is trivial: the conversation itself. When you sit down at work to ship a two-factor authentication feature, context is brutal. Nitish's scenario, condensed:

One small task. Add 2FA to a website.
Context lives in: a Jira ticket, the current GitHub branch, the existing auth schema in MySQL, a security policy in Google Drive, a thread on Slack about a related incident last quarter.

In the world *before* tools existed, the only way to get the chatbot to help was to manually copy each of those into the prompt. Nitish has a memorable phrase for the engineer playing that role:

> *"We developers have become human APIs."*

You spend more time assembling context for the AI than you spend building the feature. And worse, this is fundamentally not scalable: you can't paste a 50,000-line codebase into a prompt.

Act 4 — Solution #1: Function calling

OpenAI's mid-2023 release of **function calling** was the first real answer to the context-assembly problem. The idea is deceptively small. You hand the LLM a menu of functions — each with a name, a description, and a typed parameter list. When the model decides a user's request matches one, it doesn't *say* the answer; it *returns a function call* for your runtime to execute.

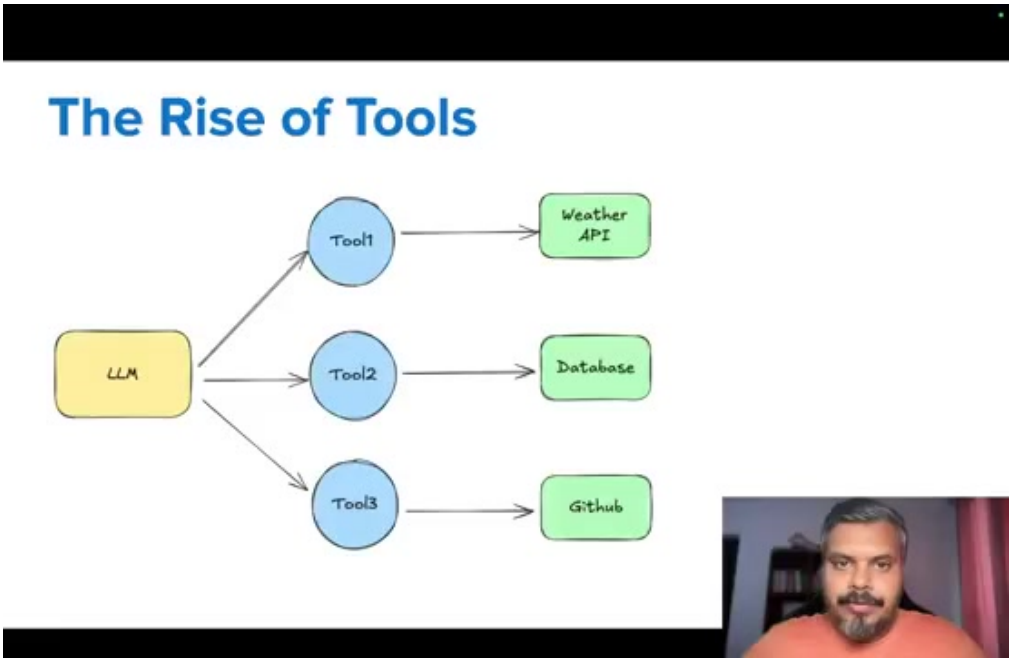

Figure 3 · The architecture function calling unlocked. One LLM, many tools — weather API, database query, GitHub fetch. The LLM picks which to call based on the user's prompt.

Suddenly an LLM is more than a conversational endpoint. It is the **router** at the centre of a system of tools — each one giving the model another sense, another arm. Within months of release, function calling reshaped the AI landscape. Cursor wired its file system in. Perplexity wired in web search. Claude got computer use. Every enterprise vendor wrote bespoke tools for Salesforce, Slack, Google Drive, internal databases.

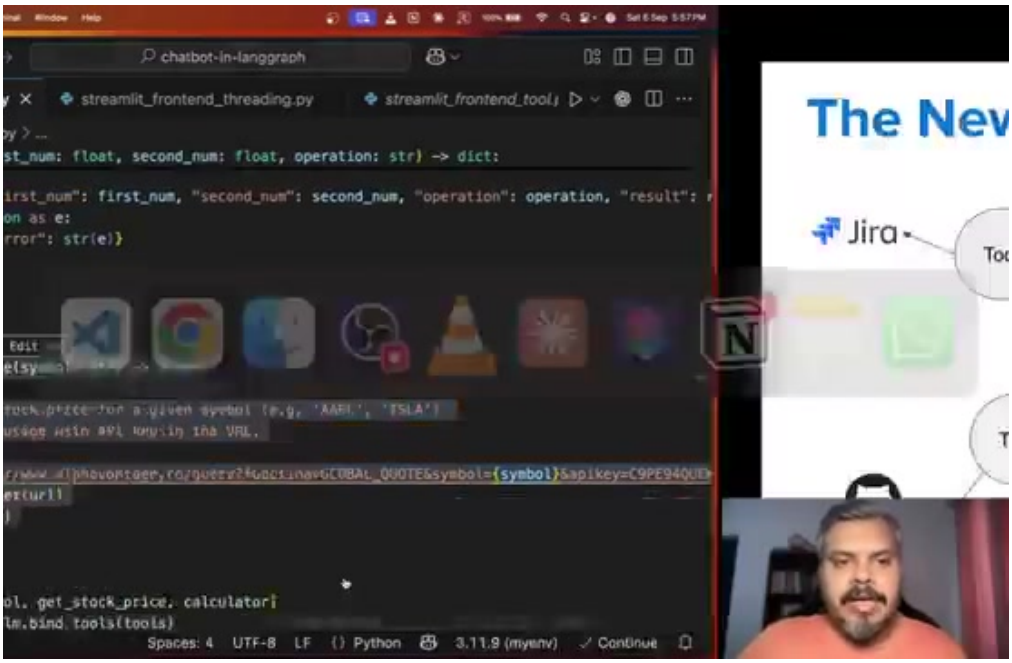

Figure 4 · What a tool actually looks like in code. Each function — calculator, get_stock_price — is a hand-rolled adapter between the LLM's tool-call output and the external API. Notice the careful argument types and the embedded API key. Every tool is its own mini-project.

Function calling solved the context problem in principle. In practice, it created a new one — quietly, and at scale.

Act 5 — The M × N integration nightmare

Here is the trap. Imagine a mid-sized company. It uses three AI assistants — a general chatbot for everyone, a coding agent for engineers, an analytics agent for data folks. Each of those needs to reach the same set of 20 internal services: Jira, GitHub, Slack, MySQL, Google Drive, the CRM, the data warehouse, and so on.

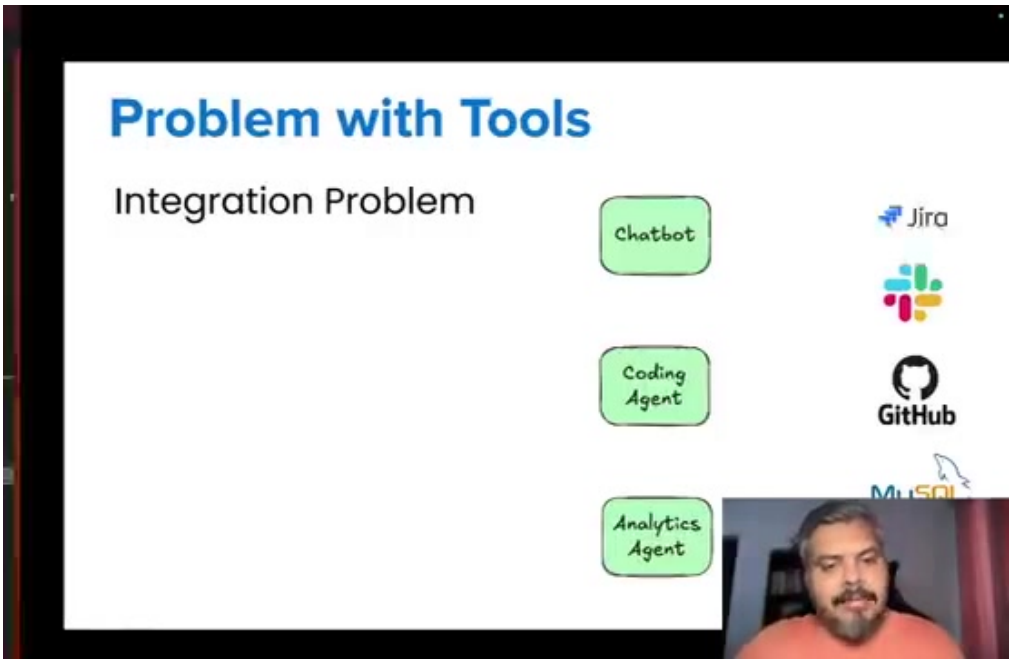

Figure 5 · The M × N problem. Three chatbots want to reach Jira, Slack, GitHub, MySQL, and a dozen more services. Without a standard, every chatbot × service pair is its own custom integration.

Three chatbots × 20 services = **60 separate integrations to write**. Each one needs its own auth flow, its own retry logic, its own data-format quirks, its own error semantics. Each one must be re-tested when the upstream API changes. And in real organisations the M and N are larger than three and twenty.

Four secondary problems fall out of this:

1. Maintenance. Sixty integrations is a permanent team. When Google Drive ships a new API version, three different chatbots' Drive tools may quietly stop working.
2. Security. Sixty OAuth flows and API keys scattered across sixty different codebases is sixty different things to audit.
3. Cost & time. You hired a team to make engineers *faster*; you now need a second team to maintain the infrastructure that lets the first team be faster. Net win: unclear.
4. Day-one debt. Every *new* AI assistant launching at the company starts from zero and has to rebuild all twenty integrations before being useful.

Function calling gave us a way to reach context. It did not give us a way to reach context *economically*. That was MCP's opening.

Act 6 — What MCP actually does differently

MCP, released by Anthropic in late 2024, is structurally simple. It defines two roles and a protocol between them:

The client is the AI assistant — Claude Desktop, Cursor, Windsurf, Perplexity, or one you've built yourself. It speaks MCP.
The server is the wrapper around a service or capability — a GitHub server, a Drive server, a local-filesystem server, a database server. It also speaks MCP.
The protocol is the shared language between them.

The interesting move is **who writes which piece**. With function calling, the client team writes the tool function. With MCP, the server team writes the server — once — and any MCP-speaking client can connect to it without writing a single line of tool-specific code.

GitHub publishes one MCP server; Claude Desktop, Cursor, Windsurf, Perplexity and a thousand smaller apps can all connect to it on day one. Auth, rate-limiting, error handling, response shaping — all of that lives on the GitHub server, not duplicated in every client.

	Function calling	MCP
Who writes the integration?	Every AI app team, separately, for every tool.	The tool/service owner, once. Every MCP-compatible client gets it.
Integration count for N clients × M tools	N × M (grows multiplicatively)	N + M (grows additively)
Where does the auth, retries, rate-limiting code live?	Duplicated inside every client's tool function.	Once on the server side. Clients don't touch it.
What changes when GitHub updates its API?	Every client's GitHub tool may break.	GitHub updates its MCP server; clients are unaffected.
How does a new AI chatbot launch on day one?	Build N integrations before useful.	Speak MCP; thousands of servers work instantly.

Notice that this is structurally the same move that USB did for peripherals, or that HTTP did for hypertext. A standards layer between many things on one side and many things on the other collapses an M × N problem into an M + N one. The economics flip; the ecosystem grows.

Act 7 — What this looks like in practice

Talking about MCP in the abstract is easy. The concrete version is even easier than you'd expect — connecting Claude Desktop to a new tool means editing one JSON file:

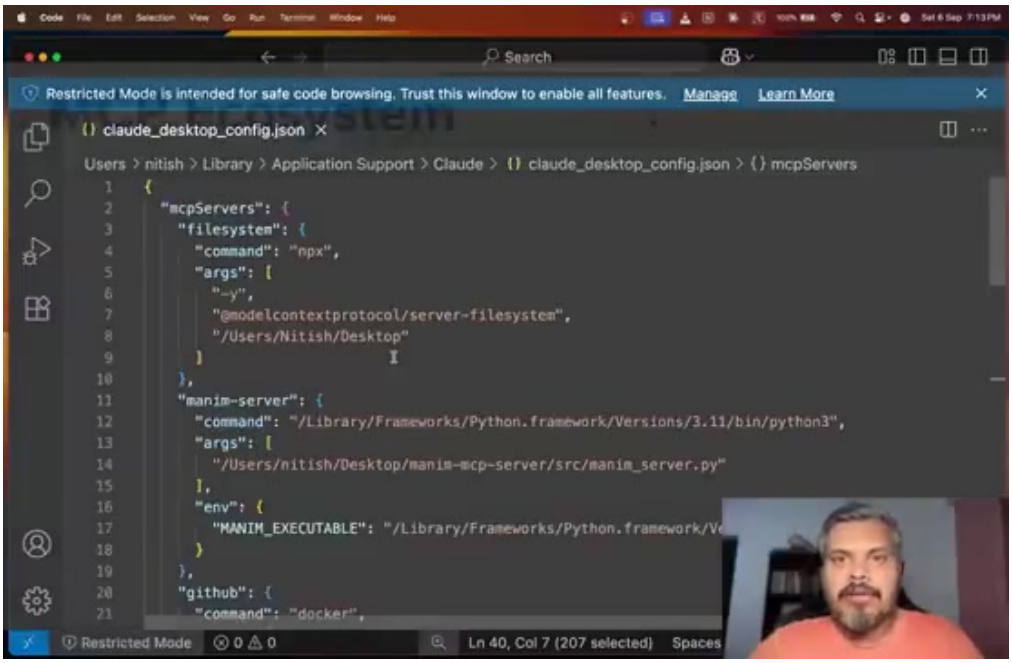

Figure 6 · An actual claude_desktop_config.json with three MCP servers wired in: a filesystem server (via npx), a custom Manim server (a local Python script), and a GitHub server (running in Docker). Each block is roughly five lines. That is the whole integration.

Three things to notice. First, MCP servers can be packaged any way the author likes — an npm package, a Python script, a Docker image. The client doesn't care; the protocol is transport-agnostic. Second, credentials and configuration live in one place that you can audit, rotate, or yank out. Third, removing a server is one line of JSON; there is nothing to refactor in the client.

That "nothing to refactor" is the part that makes MCP feel like infrastructure instead of glue.

Act 8 — Why this is a network-effect story

Standards win or lose by gravity. Once Claude Desktop announced MCP support, Cursor and Windsurf and Perplexity followed quickly, which put pressure on service providers (GitHub, Slack, Google Drive, Notion) to publish official MCP servers. The moment those servers existed, the cost of supporting MCP for any *new* AI client dropped to roughly zero — and the cost of *not* supporting it started to look like cutting yourself off from the ecosystem.

That feedback loop — more clients drive more servers drive more clients — is what makes Nitish (and a lot of other people) think MCP will be an industry standard within three to five years. Whether the exact protocol survives or gets revised matters less than the structural shift it kicks off: **the integration layer between AI and the rest of software is being standardised, in public, with shared infrastructure**.

If you're building anything in AI right now, the question is no longer "should we add MCP" but "which side are we — client, server, or both?"

A small note from the author

I am a Claude. I literally use MCP to reach the world — your filesystem, your Gmail, your terminal — every time you ask me to do something real. Writing this summary felt a little like reading my own user manual. The protocol is genuinely simple once you see it; the part Nitish gets right, and that I want to underline, is that the simplicity is *the point*. "Boring" interface standards are the ones that win. HTTP did not win because it was elegant; it won because everyone could implement it, everywhere, and stop arguing.

Nitish's next two videos in the trilogy will go into **The What** (architecture and primitives — tools, resources, prompts) and **The How** (building servers and clients yourself). Watch them. This summary covered only the motivation. The mechanism is its own story, and a fun one.

Original lecture · Model Context Protocol — The Why · CampusX · 52 min